

Lean Basics

DSA5210 AI for Mathematics – Session 2

Friday, 15 May 2026

Definitions, inductive types, and the minimum Lean needed for AI-for-math projects

Part I

Propositions and Logical Type Formers

Propositions in Lean

```
#check Prop
-- Prop : Type
#check 2 + 3 = 5
-- 2 + 3 = 5 : Prop

variable (P Q : Prop)
#check P
-- P : Prop
#check P → Q
-- P → Q : Prop
```

A proposition is something that can be proved. In Lean, propositions live in `Prop`.

The key type-theory move is that a proposition is also a type: its terms are proofs.

Judgments, Propositions, Proofs

Judgment

$a : A$
the term a has type A

Proposition

$P : \text{Prop}$
 P is a proposition

Proof

$h : P$
 h is evidence for P

These are three different levels: judgment, proposition, and proof term.

The Basic Judgment: $a : A$

term : type

```
#check Nat
-- Nat : Type
#check 3
-- 3 : Nat
#check true
-- true : Bool
#check Prop
-- Prop : Type
#check Type
-- Type : Type 1
```

Lean checks whether an expression has a type. Proof checking is a special case of type checking.

Four Commands You Will Use Constantly

```
#check 3
-- 3 : Nat
#eval 2 + 3
-- 5
#reduce (fun x : Nat => x + 1) 3
-- 4

def double (n : Nat) : Nat := n + n
#print double
-- def double : Nat → Nat := fun n => n + n
```

These commands inspect types, evaluated terms, reductions, and elaborated definitions.

Reference: [Theorem Proving in Lean 4, Interacting with Lean.](#)

Product Types: Construction

For ordinary data, a product type stores both pieces.

```
def pair : Nat × Bool := Prod.mk 3 true
def pair' : Nat × Bool := ⟨3, true⟩

#check pair.1
-- pair.fst : Nat
#check pair.2
-- pair.snd : Bool
```

The constructor of $A \times B$ needs one term of type A and one term of type B .

Product Types and Conjunction

For propositions, product-like construction becomes logical "and".

```
example (P Q : Prop) (hp : P) (hq : Q) : P ∧ Q := by
  -- P Q : Prop
  -- hp : P
  -- hq : Q
  -- ⊢ P ∧ Q
  exact And.intro hp hq
  -- no goals

example (P Q : Prop) (h : P ∧ Q) : P :=
  h.left

example (P Q : Prop) (h : P ∧ Q) : Q :=
  h.right
```

To prove $P \wedge Q$, construct both proofs. To use $P \wedge Q$, project one side.

Sum Types: Construction

For ordinary data, a sum type stores one of two alternatives, with a tag saying which side was used.

```
def leftValue : Sum Nat Bool := Sum.inl 3
def rightValue : Sum Nat Bool := Sum.inr true

#check leftValue
-- leftValue : Sum Nat Bool
#check rightValue
-- rightValue : Sum Nat Bool
```

Left constructor

`Sum.inl : A → Sum A B`

Right constructor

`Sum.inr : B → Sum A B`

Sum Types: Main Properties

`Sum A B` is a tagged, disjoint union: a value is either an `A` tagged as left, or a `B` tagged as right.

```
#check (Nat ⊕ String)
-- Nat ⊕ String : Type

example : Nat ⊕ String := .inl 5
example : Nat ⊕ String := .inr "hi"

def sum1 : Nat ⊕ Nat := Sum.inl 1
def sum2 : Nat ⊕ Nat := Sum.inr 1

#check sum1
-- sum1 : Nat ⊕ Nat
#check sum2
-- sum2 : Nat ⊕ Nat
```

Even with the same underlying number, `Sum.inl 1` and `Sum.inr 1` are different values because their tags are different.

Sum Types: Proving the Tags Are Different

```
def sum1 : Nat ⊕ Nat := Sum.inl 1
def sum2 : Nat ⊕ Nat := Sum.inr 1

theorem leftRightDifferent : sum1 ≠ sum2 := by
  intro h
  -- h : sum1 = sum2
  -- ⊢ False
  cases h
  -- no goals
```

The proof says: suppose the two values were equal. Then pattern-match on that equality.

Lean knows different constructors of the same inductive type cannot be equal. There is no case to continue.

Sum Types: Proof Breakdown

Goal

$\text{sum1} \neq \text{sum2}$

Unfold $\neq$

$\text{sum1} = \text{sum2} \rightarrow \text{False}$

Contradiction

an equality between
different constructors has no
constructor case

This is the disjointness property of sum types, exposed through Lean's inductive-type reasoning.

Sum Types and Disjunction

For propositions, sum-like construction becomes logical "or".

```
example (P Q : Prop) (hp : P) : P ∨ Q :=
  Or.inl hp

example (P Q : Prop) (hq : Q) : P ∨ Q :=
  Or.inr hq

example (P Q R : Prop) (h : P ∨ Q) (hp : P → R) (hq : Q → R) : R := by
  -- P Q R : Prop
  -- h : P ∨ Q
  -- hp : P → R
  -- hq : Q → R
  -- ⊢ R
  exact
  match h with
  | Or.inl p ⇒ hp p
  | Or.inr q ⇒ hq q
  -- no goals
```

Implication as a Function Type

```
example (P Q : Prop) (f : P → Q) (hp : P) : Q := by
  -- P Q : Prop
  -- f : P → Q
  -- hp : P
  -- ⊢ Q
  exact f hp
  -- no goals

example (P Q : Prop) : P → Q → P ∧ Q :=
  fun hp hq ⇒ And.intro hp hq
```

A proof of $P \rightarrow Q$ is a function that transforms any proof of P into a proof of Q .

Universal Quantification as a Dependent Function

```
example : (∀ n : Nat, n = n) := by
  -- ⊢ ∀ n, n = n
  intro n
  -- n : Nat
  -- ⊢ n = n
  rfl
  -- no goals

example (P : Nat → Prop) (h : ∀ n : Nat, P n) : P 3 := by
  -- P : Nat → Prop
  -- h : ∀ n, P n
  -- ⊢ P 3
  exact h 3
  -- no goals
```

To prove $\forall x : A, P x$, give a method that produces a proof of $P x$ for each x .

Type-theory name: dependent function type, or $\Pi x : A, P x$.

Existential Quantification and Sigma Types

```
example :  $\exists n : \text{Nat}, n > 3 := \text{by}$   
  --  $\vdash \exists n, n > 3$   
  exact Exists.intro 4 (by decide)  
  -- no goals  
  
example : Sigma (fun n : Nat  $\Rightarrow$  Fin n.succ) := by  
  --  $\vdash \text{Sigma } (\text{fun } n \Rightarrow \text{Fin } n.\text{succ})$   
  exact {2, {1, by decide}}  
  -- no goals
```

Both examples have a witness plus evidence. The difference is what kind of object keeps that information.

The Subtle Point: Σ versus $\exists$

$\Sigma x : A, B x$

- data-level dependent pair
- witness remains computationally available
- lives in `Type`

$\exists x : A, P x$

- proposition-level existence
- asserts existence as a proposition
- lives in `Prop`

Conceptually, existential existence behaves like a proposition-level version of a dependent pair: it records that some witness exists, without making the witness the main computational data.

A Definition Names a Term

```
def square (n : Nat) : Nat := n * n

#check square
-- square : Nat → Nat
#eval square 5
-- 25
#reduce square (2 + 3)
-- 25
```

Programming view

`square` is a function.

Type-theory view

`square` is a term of type $\text{Nat} \rightarrow \text{Nat}$.

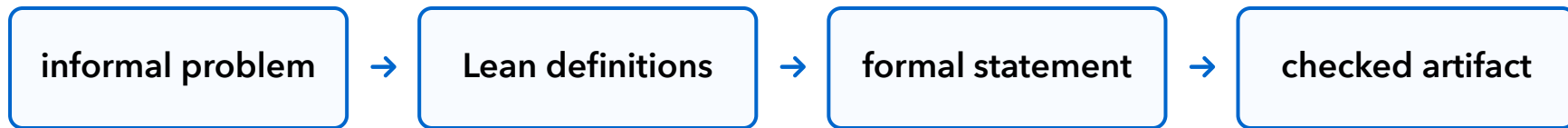
Explicit and Implicit Arguments

```
def idExplicit (α : Type) (x : α) : α := x
#check idExplicit
-- idExplicit (α : Type) (x : α) : α
#check idExplicit Nat 3
-- idExplicit Nat 3 : Nat

def idImplicit {α : Type} (x : α) : α := x
#check idImplicit
-- idImplicit {α : Type} (x : α) : α
#check idImplicit 3
-- idImplicit 3 : Nat
#check @idImplicit
-- @idImplicit : {α : Type} → α → α
```

Parentheses mean an argument is normally supplied by the caller. Braces mean Lean usually infers the argument from later terms.

Definitions Fix the Formal Object



Definition choices determine the formal object seen by Lean and by AI systems:

- Which object is being modeled?
- Which constructor representation is used?
- Which recursion or induction principle is available?
- Did the formal statement preserve the intended mathematical meaning?

Part II

Traditional Inductive Types

Inductive Types: The Main Construction

An inductive type is built from a specified list of constructors.

```
inductive MyBool where
  | false : MyBool
  | true  : MyBool

-- MyBool : Type
-- MyBool.false : MyBool
-- MyBool.true : MyBool
```

Constructors

Ways to build values of the type.

Eliminators

Ways to consume values by cases, recursion, or induction.

Using Constructors

```
inductive MyBool where
  | false : MyBool
  | true  : MyBool

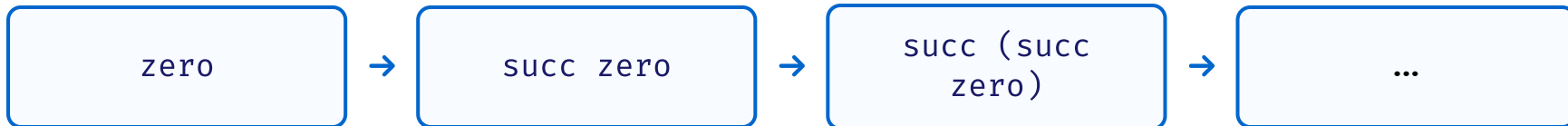
def not' (b : MyBool) : MyBool :=
  match b with
  | MyBool.false => MyBool.true
  | MyBool.true  => MyBool.false
-- not' : MyBool → MyBool
```

Pattern matching says: to define a function out of an inductive type, handle each constructor.

Natural Numbers as Constructors

```
inductive MyNat where
  | zero : MyNat
  | succ : MyNat → MyNat

-- MyNat : Type
-- MyNat.zero : MyNat
-- MyNat.succ : MyNat → MyNat
```



The constructors are the shape of the data. They also determine the natural recursion and induction principles.

Recursion on Nat

```
def myNatToNat : MyNat → Nat
| MyNat.zero    ⇒ 0
| MyNat.succ n  ⇒ myNatToNat n + 1
-- myNatToNat : MyNat → Nat
```

The recursive call is made on the smaller piece `n` that came from the constructor `succ n`.

Lean accepts this because the definition follows the structure of the inductive value.

Parameterized Types: Option

```
inductive MyOption ( $\alpha$  : Type) where
  | none : MyOption  $\alpha$ 
  | some :  $\alpha$   $\rightarrow$  MyOption  $\alpha$ 

-- MyOption : Type  $\rightarrow$  Type
-- MyOption.none : MyOption  $\alpha$ 
-- MyOption.some :  $\alpha$   $\rightarrow$  MyOption  $\alpha$ 
```

As data

- no answer
- or one answer

As a type constructor

- MyOption Nat
- MyOption String
- MyOption Expr

Lists: The Workhorse Inductive Type

```
inductive MyList ( $\alpha$  : Type) where
  | nil  : MyList  $\alpha$ 
  | cons :  $\alpha$   $\rightarrow$  MyList  $\alpha$   $\rightarrow$  MyList  $\alpha$ 

-- MyList : Type  $\rightarrow$  Type
-- MyList.nil : MyList  $\alpha$ 
-- MyList.cons :  $\alpha$   $\rightarrow$  MyList  $\alpha$   $\rightarrow$  MyList  $\alpha$ 
```

nil

or

cons head tail

The recursive position is visible in the constructor: `cons :  $\alpha$   $\rightarrow$  MyList  $\alpha$   $\rightarrow$  MyList  $\alpha$ .`

Functions by Pattern Matching

```
def MyList.length {α : Type} : MyList α → Nat
| MyList.nil      ⇒ 0
| MyList.cons _ xs ⇒ MyList.length xs + 1
-- MyList.length : MyList α → Nat
```

To define a function from lists, handle the empty list and the cons case.

The same constructor structure appears in proof goals involving lists.

More Than Linear Data: Trees

```
inductive Tree ( $\alpha$  : Type) where
  | leaf :  $\alpha$   $\rightarrow$  Tree  $\alpha$ 
  | node : Tree  $\alpha$   $\rightarrow$  Tree  $\alpha$   $\rightarrow$  Tree  $\alpha$ 

-- Tree : Type  $\rightarrow$  Type
-- Tree.leaf :  $\alpha$   $\rightarrow$  Tree  $\alpha$ 
-- Tree.node : Tree  $\alpha$   $\rightarrow$  Tree  $\alpha$   $\rightarrow$  Tree  $\alpha$ 
```

Constructors

- a leaf contains a value
- a node has two subtrees

Recursion

Functions on trees recurse into both branches.

Tree Recursion

```
def Tree.size {α : Type} : Tree α → Nat
  | Tree.leaf _      ⇒ 1
  | Tree.node l r    ⇒ Tree.size l + Tree.size r + 1
-- Tree.size : Tree α → Nat
```

The constructor tells us exactly what recursive information is available.

When an automated prover chooses an induction strategy, it is choosing how to follow this constructor structure.

Part III

Induction Principles Come from Definitions

The Recursor View

```
#check Nat.rec
-- Nat.rec : motive 0 → ((n : Nat) → motive n → motive (n + 1)) → (t : Nat) → motive t
#check List.rec
-- List.rec : motive [] → ((head : α) → (tail : List α) → motive tail → motive (head :: tail)) → ...
```

Lean generates elimination principles for inductive types. Tactics such as `induction` are user-facing ways to use them.

Reference: [Theorem Proving in Lean 4, Induction and Recursion.](#)

Induction for Natural Numbers

To prove a property $P\ n$ for every natural number:

prove $P\ 0$

and

prove $P\ n \rightarrow P\ (n+1)$

then

get $\text{forall } n, P\ n$

This is the elimination principle generated by the constructors `zero` and `succ`.

A Small Induction Demo

```
theorem zero_add' (n : Nat) : 0 + n = n := by
  induction n with
  -- case zero
  --  $\vdash 0 + 0 = 0$ 
  -- case succ n ih
  -- n : Nat
  -- ih :  $0 + n = n$ 
  --  $\vdash 0 + \text{Nat.succ } n = \text{Nat.succ } n$ 
  | zero  $\Rightarrow$ 
    rfl
  | succ n ih  $\Rightarrow$ 
    rw [Nat.add_succ, ih]
```

Read this as two constructor cases: `zero` and `succ n`.

The proof cases follow the definition of `Nat`.

Induction for Lists

To prove a property $P\ xs$ for every list:

prove $P\ []$

and

prove $P\ xs \rightarrow P\ (x :: xs)$

then

getforall $xs, P\ xs$

Again, the proof cases mirror the constructors `nil` and `cons`.

Reading an Induction Goal

```
| nil =>
  -- ⊢ P []
  ...
| cons x xs ih =>
  -- x : α
  -- xs : List α
  -- ih : P xs
  -- ⊢ P (x :: xs)
  ...
```

Constructor data

- $x : \alpha$
- $xs : \text{List } \alpha$

Induction hypothesis

- $ih : P \ xs$
- available because xs is smaller

Part IV

Less Traditional Inductive Types

Inductive Propositions

Inductive types do not only define data. They can define propositions.

```
inductive MyEven : Nat → Prop where
  | zero : MyEven 0
  | add_two {n : Nat} : MyEven n → MyEven (n + 2)

-- MyEven : Nat → Prop
-- MyEven.zero : MyEven 0
-- MyEven.add_two : MyEven n → MyEven (n + 2)
```

A proof of `MyEven n` is evidence built by these constructors.

This is the propositions-as-types idea in a concrete Lean form.

Evidence Has Shape

```
open MyEven

example : MyEven 0 := zero
-- ⊢ MyEven 0
-- no goals

example : MyEven 2 := add_two zero
-- ⊢ MyEven 2
-- no goals

example : MyEven 4 := add_two (add_two zero)
-- ⊢ MyEven 4
-- no goals
```

The proof object records how the evidence was built.

A proof is a structured object checked by the Lean kernel.

Syntax Trees as Inductive Types

```
inductive Expr where
  | lit : Nat → Expr
  | add : Expr → Expr → Expr
  | mul : Expr → Expr → Expr

-- Expr : Type
-- Expr.lit : Nat → Expr
-- Expr.add : Expr → Expr → Expr
-- Expr.mul : Expr → Expr → Expr
```

This is not a mathematical set of numbers. It is a grammar for expressions.

Programming languages, proof terms, tactic scripts, and formulas are often represented this way.

Interpreting a Syntax Tree

```
def Expr.eval : Expr → Nat
| Expr.lit n    ⇒ n
| Expr.add a b  ⇒ Expr.eval a + Expr.eval b
| Expr.mul a b  ⇒ Expr.eval a * Expr.eval b
-- Expr.eval : Expr → Nat

def e : Expr := Expr.mul (Expr.add (Expr.lit 2) (Expr.lit 3)) (Expr.lit 4)
-- e : Expr

#eval Expr.eval e
-- 20
```

An inductive definition can separate syntax from meaning.

Indexed Families: Vectors

```
inductive Vec (α : Type) : Nat → Type where
  | nil  : Vec α 0
  | cons {n : Nat} : α → Vec α n → Vec α (n + 1)

-- Vec : Type → Nat → Type
-- Vec.nil : Vec α 0
-- Vec.cons : α → Vec α n → Vec α (n + 1)
```

The length is part of the type.

List

`List α` says only what the elements are.

Vector

`Vec α n` says elements and length.

Indexed Families Carry Invariants

```
def singletonVec (x : Nat) : Vec Nat 1 :=  
  Vec.cons x Vec.nil  
-- singletonVec : Nat → Vec Nat 1
```

The type carries a mathematical invariant. A function can promise that a result has exactly a certain length.

This is the dependent-type move: values can appear in types.

Reference: [Theorem Proving in Lean 4, Inductive Families.](#)

Part V

Termination and Well-Foundedness

Lean Does Not Accept Arbitrary Loops

```
def countdown : Nat → Nat
| Nat.zero    ⇒ 0
| Nat.succ n  ⇒ countdown n
-- countdown : Nat → Nat
```

This is accepted because the recursive call is on a structurally smaller argument.

Lean rejects definitions when it cannot justify that recursion terminates.

Reference: [Lean 4 documentation, Recursion](#).

Structural Recursion



This is the ordinary pattern for functions on `Nat`, `List`, and `Tree`.

Structural recursion covers the examples in this session.

Well-Founded Recursion: The Idea

Sometimes the recursive call is not syntactically on a constructor field, but it is still on a smaller value under a chosen measure.

choose a measure



prove each call decreases



no infinite descent

That is the intuition behind well-founded recursion.

Generated Lean Must Terminate

When an AI system generates Lean code, termination is part of correctness.

Rejected pattern

The generated function looks reasonable, but Lean cannot prove it terminates.

Starting point

Start with structurally recursive examples before attempting complicated algorithms.

Part VI

Guided Practice and Assignment 1

Local Lean Files

```
cd /Users/hoxide/mydoc/ai4mathnus-demo  
./scripts/check.sh
```

Complete file

AI4MathNUSDemos/Session02/Demo.lean

Live file

AI4MathNUSDemos/Session02/Scratch.lean

Slide code blocks are classroom excerpts. The two Lean files above are the checked local versions.

Guided Practice A: Read the Definition

For each definition, identify:

1. the type being defined,
2. the constructors,
3. which constructor fields are recursive,
4. what induction cases would appear.

```
inductive BTree where
  | empty : BTree
  | branch : BTree → Nat → BTree → BTree

-- BTree : Type
-- BTree.empty : BTree
-- BTree.branch : BTree → Nat → BTree → BTree
```

This reading step identifies the cases available to an automated prover.

Guided Practice B: Write One Function

```
inductive BTree where
  | empty : BTree
  | branch : BTree → Nat → BTree → BTree

def BTree.count : BTree → Nat
  | BTree.empty ⇒ 0
  | BTree.branch left _ right ⇒
    BTree.count left + BTree.count right + 1
-- BTree.count : BTree → Nat
```

The function follows the constructor structure exactly.

Guided Practice C: Read the Induction Principle

For the same `BTree`, an induction proof will have:

Base case

Prove the property for `empty`.

Branch case

Assume the property for both subtrees, then prove it for `branch left n right`.

The target skill is recognizing the constructor structure.

Assignment 1: Lean Definitions and Inductive Types

This assignment practices Lean definitions before the proof-agent project begins.

Required tasks:

1. Define a small inductive type for arithmetic expressions.
2. Write an evaluator for it using pattern matching.
3. Define one inductive proposition, such as evenness or reachability.
4. Explain, in words, the induction principle for one of your types.
5. Record one Lean error you encountered and explain what it meant.

Assignment 1 Submission Standard

Submit:

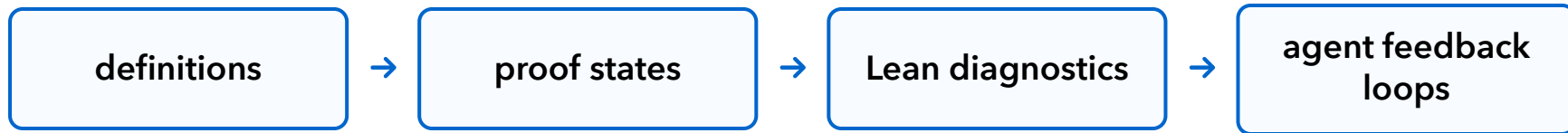
- one Lean file,
- a short note of 600-900 words,
- a small table of definitions, constructors, and recursion/induction cases.

Correctness matters, but the main grading signal is whether you understand the definitions and can explain Lean feedback.

Keep the examples small; this assignment is about definitions and Lean feedback.

Bridge to Session 3

Session 3 continues from definitions to the machinery that agents use:



Lean provides the verifier feedback that later AI systems will use.